

AI 바이브코딩 관련 대화

Parquet(파케이)는 대용량 데이터 처리를 위해 최적화된 **컬럼 기반(Columnar)** 오픈 소스 데이터 저장 포맷입니다. Apache Hadoop 에코시스템을 비롯하여 Spark, 대용량 데이터웨어하우스(DW) 등 빅데이터 분석 영역에서 표준처럼 널리 사용되고 있습니다.

Parquet 포맷이 무엇인지 이해하기 위해 핵심 특징과 장점을 단계별로 나누어 설명해 드리겠습니다.

1. 행(Row) 기반 vs 컬럼(Column) 기반 저장 방식

전통적인 데이터베이스나 CSV, JSON 등은 행 기반(Row-oriented)으로 데이터를 저장합니다. 반면 Parquet은 컬럼 기반(Columnar)으로 데이터를 저장합니다.

- **행 기반 (CSV, JSON 등):** [1, 홍길동, 30], [2, 김철수, 25]와 같이 한 줄(레코드)씩 모아서 저장합니다. 특정 사용자의 정보를 추가하거나 전체를 읽을 때 유리합니다.
- **컬럼 기반 (Parquet):** [1, 2] (ID 컬럼), [홍길동, 김철수] (이름 컬럼), [30, 25] (나이 컬럼)과 같이 동일한 데이터 타입의 컬럼끼리 모아서 저장합니다.

2. Parquet의 주요 장점

효율한 데이터 압축률

동일한 컬럼 내에는 항상 같은 데이터 타입(예: 나이 컬럼에는 숫자만, 이름 컬럼에는 문자열만)이 존재합니다. 데이터 타입이 같으면 압축 알고리즘(Gzip, Snappy, Zstd 등)의 효율이 극대화되기 때문에, CSV 파일에 비해 용량을 획기적으로 줄일 수 있습니다. 보통 CSV 대비 75% 이상 용량이 줄어듭니다.

디스크 I/O 및 쿼리 속도 최적화 (프로젝션 효율성)

만약 수백 개의 컬럼을 가진 기가바이트 단위의 테이블에서 '나이' 컬럼의 평균값만 구하는 쿼리를 실행한다고 가정해 보겠습니다.

- **행 기반:** 원하지 않는 다른 컬럼 데이터까지 디스크에서 전부 읽어와야 하므로 불필요한 디스크 I/O가 대량으로 발생합니다.
- **컬럼 기반(Parquet):** 디스크에서 오직 '나이' 컬럼이 저장된 구역만 찾아서 읽기 때문에 처리 속도가 압축적으로 빨라집니다.

조건절 푸시다운 (Predicate Pushdown)

Parquet 파일 내부에는 메타데이터가 함께 저장됩니다. 이 메타데이터에는 각 데이터 블록의 컬럼별 최솟값(Min)과 최댓값(Max) 등의 통계 정보가 포함되어 있습니다.

쿼리에서 WHERE 나이 > 50과 같은 조건을 걸면, 메타데이터를 먼저 확인하여 조건에

맞지 않는 데이터 블록은 아예 읽지 않고 건너뛰(Skip)니다. 이를 통해 스캔하는 데이터 양을 최소화합니다.

명확한 스키마 정의

CSV는 각 컬럼이 숫자 형태인지 문자열 형태인지 모호하여 파싱할 때 오류가 자주 발생합니다. Parquet은 파일 자체에 데이터 타입(Schema) 정보가 명확히 기록되어 있어 데이터 오염을 방지합니다.

3. 요약

Parquet 포맷은 "자주 조회하는 특정 컬럼만 빠르게 골라내어 분석"하거나 **"용량을 대폭 줄여서 저장 비용을 아끼고자 할 때"** 사용하는 대용량 데이터 분석용 최적화 파일 포맷입니다.

단, 데이터를 한 줄씩 실시간으로 자주 수정하거나 추가(Insert/Update)해야 하는 트랜잭션 중심의 작업에는 적합하지 않으며, 한 번 대량으로 쓰여진 데이터를 반복적으로 읽고 분석하는 작업(Write Once, Read Many)에 가장 강력한 성능을 발휘합니다.

Git에서 스테이징 영역(Staging Area)과 로컬 저장소(Local Repository)는 버전 관리를 수행하는 과정에서 서로 다른 목적과 역할을 가진 공간입니다. Git의 작업 흐름을 이해하는 데 핵심이 되는 두 영역의 차이점을 단계별로 나누어 설명해 드리겠습니다.

1. 개념의 정의

스테이징 영역 (Staging Area / Index)

- **역할:** 다음 커밋(Commit)에 포함할 변경 사항들을 임시로 저장하고 준비하는 완충 지대입니다.
- **특징:** 작업 디렉터리(Working Directory)에서 수정된 파일 중, git add 명령어를 통해 선택된 파일들만 이 영역에 올라갑니다. 파일의 스냅샷이 커밋되기 직전의 상태로 대기하는 곳입니다.

로컬 저장소 (Local Repository)

- **역할:** 제출된 파일들의 버전(커밋) 히스토리가 영구적으로 안전하게 기록되는 공간입니다.
- **특징:** git commit 명령어를 실행하면 스테이징 영역에 있던 변경 사항들이 하나의 버전(커밋)으로 묶여 로컬 저장소에 저장됩니다. 사용자의 컴퓨터 내부(.git 디렉터리)에 존재하며, 오프라인 상태에서도 전체 히스토리를 조회할 수 있습니다.

2. 작업 흐름에서의 차이 (단계별 비교)

Git은 파일을 관리할 때 일반적으로 3단계의 논리적 공간을 거치게 됩니다.

1. **작업 디렉터리 (Working Directory):** 실제로 코드를 수정하고 파일을 생성하는 컴퓨터의 디렉터리 공간입니다.
2. **스테이징 영역 (Staging Area):** 변경된 파일 중 "이 파일들을 다음 버전에 기록하겠다"라고 선택하여 올려두는 공간입니다 (git add).
3. **로컬 저장소 (Local Repository):** 선택된 변경 사항들을 "하나의 버전으로 확정하여 기록하겠다"라고 선언하여 저장하는 공간입니다 (git commit).

3. 왜 스테이징 영역이 따로 필요할까?

로컬 저장소로 바로 커밋하지 않고 중간에 스테이징 영역을 두는 이유는 **커밋의 단위를 논리적으로 깔끔하게 분리하기 위해서**입니다.

- **선택적 커밋 가능:** 작업 디렉터리에서 파일 5개를 수정했지만, 그중 기능 A와 관련된 파일 3개만 먼저 하나의 버전으로 만들고 싶을 때가 있습니다. 이때 관련된 3개의 파일만 스테이징 영역에 올려(git add) 커밋을 생성하면, 의미가 명확하게 분리된 히스토리를 유지할 수 있습니다.
- **충돌 및 검토 완충지대:** 커밋을 하기 전에 git diff --staged 같은 명령어를 통해

내가 이번 버전에 무엇을 저장하려고 하는지 최종적으로 검토할 수 있는 기회를 제공합니다.

4. 핵심 명령어 및 상태 비교

두 영역의 차이를 명령어를 통해 직관적으로 비교하면 다음과 같습니다.

- **작업 디렉터리 → 스테이징 영역:** `git add <파일명>`
- **스테이징 영역 → 로컬 저장소:** `git commit -m "커밋 메시지"`
- **상태 확인:**
 - `git status`를 입력했을 때 **초록색**으로 표시되는 파일은 스테이징 영역에 있는 상태입니다.
 - **빨간색**으로 표시되는 파일은 작업 디렉터리에만 있고 아직 스테이징 영역에 올라가지 않은 상태입니다.

Windows 운영체제를 기준으로 설명해 드리면, 스테이징 영역과 로컬 저장소는 별도의 프로그램이나 외부 서버가 아니라 **사용자가 프로젝트를 진행하는 폴더 내부의 숨겨진 디렉터리** 안에 물리적으로 생성됩니다.

구체적으로 어떤 경로에 어떻게 존재하는지 단계별로 나누어 설명해 드리겠습니다.

1. 실제 생성 위치: .git 폴더

Git을 사용하기 위해 특정 폴더에서 git init 명령어를 실행하면, 해당 폴더 내부에 **.git이라는 이름의 숨김 폴더**가 자동으로 생성됩니다. 이 .git 폴더 내부가 바로 Git의 모든 마법이 일어나는 공간입니다.

Windows 탐색기 기준으로 경로는 다음과 같습니다.

C:\Users\사용자이름\Documents\프로젝트폴더\.git\

**참고: Windows에서는 이름 앞에 점(.)이 붙은 폴더나 파일은 기본적으로 숨김 처리되어 보이지 않을 수 있습니다. 파일 탐색기 상단의 [보기] -> [표시] -> [숨긴 항목]*에 체크하시면 확인할 수 있습니다.*

2. 스테이징 영역과 로컬 저장소의 물리적 파일 구조

.git 폴더 내부를 열어보면 여러 파일과 폴더가 존재하는데, 스테이징 영역과 로컬 저장소는 각각 아래와 같은 파일과 폴더로 매핑되어 관리됩니다.

스테이징 영역 (Staging Area)의 위치

- **물리적 형태:** .git 폴더 바로 아래에 생성되는 index라는 이름의 바이너리 파일입니다.
- **경로:** .../프로젝트폴더/.git/index
- **설명:** 사용자가 git add 명령어를 실행하면, 변경된 파일의 정보(파일명, 파일 경로, 파일 내용의 해시값 등)가 이 index 파일 안에 기록됩니다. 즉, 스테이징 영역은 독립된 폴더가 아니라 하나의 파일 형태로 관리되는 가상의 준비 공간입니다.

로컬 저장소 (Local Repository)의 위치

- **물리적 형태:** .git 폴더 내부에 있는 objects 폴더와 refs 폴더입니다.
- **경로:**
 - .../프로젝트폴더/.git/objects/ (실제 커밋된 파일의 압축 데이터 저장)
 - .../프로젝트폴더/.git/refs/ (브랜치 및 태그 정보 저장)
- **설명:** 사용자가 git commit 명령어를 실행하면, index(스테이징 영역)에 있던 스냅샷을 토대로 영구적인 고유 해시값을 가진 파일 객체들이 objects 폴더

내부에 생성됩니다. 이 공간이 바로 버전 히스토리가 안전하게 누적되는 실제 로컬 저장소입니다.

3. 요약

Windows 환경에서 두 영역은 다음과 같이 요약할 수 있습니다.

- **스테이징 영역:** `.git/index` 파일로 존재함 (다음 커밋에 들어갈 목록을 들고 있는 명세서 역할)
- **로컬 저장소:** `.git/objects/` 및 `.git/refs/` 폴더로 존재함 (실제 커밋된 데이터와 버전 기록이 쌓이는 금고 역할)

따라서 이 `.git` 폴더만 통째로 복사하면 다른 컴퓨터로 옮겨도 지금까지의 모든 커밋 히스토리와 스테이징 상태를 그대로 유지할 수 있습니다. 반대로 이 폴더를 삭제하면 해당 프로젝트의 모든 Git 버전 관리 기록이 사라지게 됩니다.

결론부터 말씀드리면, `git add .` 명령은 변경이 있는 파일들을 '커밋할 준비 상태(Staging Area)'로 만드는 것이지, 실제로 커밋을 수행하는 것은 아닙니다. Git의 작동 구조와 해당 명령의 정확한 역할을 단계별로 나누어 설명해 드리겠습니다.

1. Git의 3가지 공간 (Working Directory, Staging Area, Repository)

Git은 소스코드를 관리할 때 크게 3가지 공간을 거치게 됩니다.

- **Working Directory (작업 디렉토리):** 사용자가 실제로 파일을 수정, 생성, 삭제하는 PC의 실제 폴더 공간입니다.
- **Staging Area (스테이징 영역):** 다음 커밋에 포함할 파일들을 임시로 모아두는 준비 공간입니다.
- **Repository (저장소):** `git commit`을 통해 스냅샷이 영구적으로 저장되는 공간입니다.

`git add .`는 파일의 상태를 **Working Directory**에서 **Staging Area**로 이동시키는 역할을 합니다.

2. git add . 의 정확한 역할

`git add .` 명령은 **현재 디렉토리**와 그 하위 디렉토리 내에서 **변경이 발생한 모든 파일**을 Staging Area에 올립니다. 여기서 '변경'이란 다음 3가지를 모두 포함합니다.

- 새롭게 생성된 파일 (Untracked files)
- 기존에 존재하던 파일 중 내용이 수정된 파일 (Modified files)
- 기존에 존재하던 파일 중 삭제된 파일 (Deleted files)

주의할 점: (점)은 현재 디렉토리를 의미하므로, 만약 상위 디렉토리에 변경사항이 있더라도 현재 명령을 실행하는 위치가 하위 폴더 내부라면 상위 폴더의 변경사항은 포함되지 않습니다. 프로젝트 전체의 변경사항을 위치에 상관없이 모두 올리고 싶다면 `git add -A` 또는 `git add --all`을 사용하는 것이 더 안전합니다.

3. 커밋까지 완료하려면?

`git add .`를 수행한 후, 이 변경사항을 최종적으로 저장소에 기록(커밋)하려면 반드시 `git commit` 명령을 추가로 실행해야 합니다. 보통은 다음과 같이 두 명령을 연속해서 사용합니다.

Bash

```
git add .
```

```
git commit -m "커밋 메시지 작성"
```

만약 `git add` 단계를 생략하고 변경된 파일(수정 및 삭제)을 바로 커밋하고 싶다면

아래와 같이 -a 옵션을 사용할 수도 있습니다. 다만 이 방법은 새롭게 생성된 파일(Untracked)은 인식하지 못하므로 상황에 맞게 주의해서 사용해야 합니다.

Bash

```
git commit -am "수정 및 삭제된 파일 바로 커밋"
```

네, 그러한 프로그램은 **100% 가능하며, 실제로 많은 개인 투자자들과 개발자들이 애용하는 방식**입니다.

한국투자증권은 'KIS Developers'라는 오픈 API 포털을 매우 적극적으로 운영하고 있어 시세 조회뿐만 아니라 실제 매매 주문까지 파이썬 코드로 제어할 수 있습니다. 카카오톡 역시 'Kakao Developers'를 통해 본인 스마트폰으로 메시지를 보내는 '나에게 보내기 API'를 공식 제공합니다.

이 프로그램을 개발하기 위한 구조와 단계별 절차, 그리고 바로 실행해서 구조를 이해할 수 있는 전체 파이썬 예제 코드를 안내해 드리겠습니다.

1. 프로그램 개발 로드맵

1 단계: 한국투자증권 오픈 API 준비

1. 한국투자증권 'KIS Developers' 사이트에 접속하여 로그인합니다.
2. 실전투자 혹은 모의투자 **AppKey**와 **AppSecret**을 발급받습니다.
3. API 호출에 필요한 웹 토큰(Access Token) 발급 방식을 숙지합니다.

2 단계: 카카오톡 메시지 API 준비

1. 'Kakao Developers' 사이트에 접속하여 애플리케이션을 생성합니다.
2. [카카오 로그인] 메뉴에서 활성화 상태를 ON으로 바꾸고, Redirect URI(예: <https://localhost:5000>)를 등록합니다.
3. [동의항목] 메뉴에서 '**카카오톡 메시지 전송**' 권한을 '선택 동의'로 설정합니다.
4. 인증을 거쳐 카카오톡 API 전송용 Access Token을 발급받습니다.

3 단계: 모니터링 및 조건 로직 구현

1. 파이썬의 `time.sleep()` 함수나 스케줄러를 활용해 주기적으로 한국투자증권 API를 호출하여 특정 종목의 현재가를 가져옵니다.
2. 지정한 조건(예: 특정 가격 이하로 하락, 혹은 특정 가격 이상으로 상승)을 만족하는지 체크합니다.
3. 조건을 만족하면 카카오톡 API를 호출하여 메시지를 전송합니다.

2. 모니터링 및 카카오톡 전송 프로그램 전체 예제 코드

아래 코드는 API 호출 및 토큰 발급의 전체적인 흐름을 이해하고, 바로 로직 테스트를 해볼 수 있도록 구성한 통합 예제입니다. 한국투자증권과 카카오톡 API는 보안상 토큰이 주기적으로 만료되므로, 실제 운영 환경에서는 자동 갱신 로직이 포함되지만 구조 이해를 위해 핵심 흐름 위주로 작성되었습니다.

Python

```

import json

import time

import requests

# =====

# 1. 설정 정보 입력 (발급받은 실제 키로 대체)

# =====

# 한국투자증권 API 설정
KIS_APP_KEY = "your_kis_app_key"
KIS_APP_SECRET = "your_kis_app_secret"
KIS_URL = "https://openapi.koreainvestment.com:9443" # 실전투자 서버 (모의투자는 다름)

# 카카오톡 API 설정
KAKAO_ACCESS_TOKEN = "your_kakao_access_token"

# 모니터링 대상 설정
TARGET_STOCK_CODE = "005930" # 삼성전자 종목코드
TARGET_PRICE = 70000 # 알림을 받을 목표 가격 조건

# =====

# 2. 기능별 함수 구현

# =====

def get_kis_access_token():
    """한국투자증권 API 접근을 위한 토큰 발급"""
    url = f"{KIS_URL}/oauth2/tokenP"
    headers = {"content-type": "application/json"}

```

```
payload = {  
    "grant_type": "client_credentials",  
    "appkey": KIS_APP_KEY,  
    "appsecret": KIS_APP_SECRET,  
}  
  
try:  
    # 실제 호출 시 주석 해제하여 사용  
    # response = requests.post(url, headers=headers, data=json.dumps(payload))  
    # return response.json()["access_token"]  
  
    # 구조 이해를 위한 가상 토큰 반환  
    return "mock_kis_token_abcdefg"  
except Exception as e:  
    print(f"한투 토큰 발급 실패: {e}")  
    return None  
  
def get_current_price(stock_code, kis_token):  
    """한국투자증권 오픈 API 를 이용한 특정 종목 현재가 조회"""  
    url = f"{KIS_URL}/uapi/domestic-stock/v1/quotations/inquire-price"  
  
    # 한국투자증권에서 요구하는 표준 헤더 구성  
    headers = {  
        "content-type": "application/json",  
        "authorization": f"Bearer {kis_token}",  
        "appkey": KIS_APP_KEY,
```

```

"appsecret": KIS_APP_SECRET,

"tr_id": "FHKST01010100", # 주식현재가 조회용 TR ID
}

params = {
    "FID_COND_MRKT_DIV_CODE": "J", # 주식 시장 구분 (J: 주식)
    "FID_INPUT_ISCD": stock_code, # 종목코드
}

try:
    # 실제 호출 시 주식 해제하여 사용
    # response = requests.get(url, headers=headers, params=params)
    # return int(response.json()["output"]["stck_prpr"])

    # 실습을 위해 실시간 변동성이 있는 가상의 가격을 반환 (조금씩 하락하는 상황 가정)
    import random

    global mock_base_price
    mock_base_price -= random.randint(200, 500)
    return mock_base_price
except Exception as e:
    print(f"시세 조회 실패: {e}")
    return None

def send_kakao_message(text_message):
    """카카오톡 '나에게 보내기' API 를 사용한 메시지 전송"""

```

```

url = "https://kapi.kakao.com/v2/api/talk/memo/default/send"

headers = {"Authorization": f"Bearer {KAKAO_ACCESS_TOKEN}"}

# 카카오톡이 요구하는 템플릿 구조 정의
template_object = {
    "object_type": "text",
    "text": text_message,
    "link": {
        "web_url": "https://apiportal.koreainvestment.com",
        "mobile_web_url": "https://apiportal.koreainvestment.com",
    },
    "button_title": "한투 개발자센터 이동",
}

payload = {"template_object": json.dumps(template_object)}

try:
    # 실제 호출 시 주석 해제하여 사용
    # response = requests.post(url, headers=headers, data=payload)

    # if response.status_code == 200:
    #     print("카카오톡 메시지 전송 성공")
    # else:
    #     print(f"카카오톡 전송 실패: {response.json()}")

    print(f"\n[카카오톡 발송 가상 알림]\n 메시지 내용: {text_message}")

    return True
except Exception as e:

```

```

        print(f"카카오 API 호출 에러: {e}")

        return False

# =====
# 3. 메인 모니터링 루프 실행
# =====

if __name__ == "__main__":

    print("주식 시세 모니터링 프로그램을 시작합니다.")

    # 실습용 가상 시작가 설정 (조건 만족 테스트용)
    mock_base_price = 71000

    # 1. 한국투자증권 토큰 확보

    kis_token = get_kis_access_token()

    if kis_token:

        # 2. 실시간 감시 시작 (조건이 만족될 때까지 반복 수집)

        alert_sent = False

        monitor_count = 0

        while not alert_sent and monitor_count < 5:

            monitor_count += 1

            current_price = get_current_price(TARGET_STOCK_CODE, kis_token)

            if current_price:

                print(

```

```

        f"[{monitor_count}회차 주식 감시] 현재가: {current_price;}원 (목표가:
{TARGET_PRICE;}원)"

    )

    # 조건 비교: 현재가가 목표가 이하로 떨어졌을 때 알림

    if current_price <= TARGET_PRICE:

        msg = f"🚨 알림! 종목코드[{TARGET_STOCK_CODE}] 현재가가
{current_price;}원으로 설정하신 목표가 {TARGET_PRICE;}원 이하로 도달했습니다."

        send_kakao_message(msg)

        alert_sent = True # 알림을 보냈으므로 프로그램 종료 조건 만족

    else:

        print("-> 아직 조건에 도달하지 않았습니다. 2 초 후 다시 확인합니다.")

    time.sleep(2) # 과도한 API 호출 방지를 위한 시간 지연

    if not alert_sent:

        print("테스트 제한 횟수 도달로 모니터링을 종료합니다.")

```

3. 구현 시 주의해야 할 점

1. **카카오톡 토큰의 만료 시간:** 카카오톡 Access Token 은 유효기간이 수 시간(보통 6 시간)으로 짧습니다. 장시간 백그라운드로 켜놓는 프로그램을 만들 때는 만료되기 전에 Refresh Token 을 사용해 Access Token 을 재발급받는 로직이 최종적으로 추가되어야 합니다.
2. **트래픽 제한(Rate Limit):** 증권사 API 는 초당 호출 횟수 제한(예: 초당 3~5 회)이 있습니다. 반복문 내부에서 조건 없이 너무 빠르게 요청을 날리면 IP 가 차단될 수 있으므로 반드시 호출 사이에 적절한 대기 시간(time.sleep)을 할당해야 합니다. 실시간에 가까운 체결 데이터를 얻고 싶다면 대안으로 제공되는 한국투자증권 **웹소켓(WebSocket) API** 를 연동하는 방식으로 확장하시면 됩니다.

네, 충분히 가능합니다! 이전에 KakaoTalk 알림 시스템 검토하셨던 것과 딱 맞는 조합이네요.

구성 요소

한국투자증권 KIS Open API 카카오톡 알림
(시세 모니터링 + 조건 감지) →→→ (나에게 메시지 전송)
두 API 모두 개인 무료로 사용 가능합니다.

1. 한국투자증권 KIS Open API

- REST API 방식, Python 으로 간단히 호출
- 무료, 개인 계좌만 있으면 신청 가능
- 실시간 시세, 일봉/분봉, 잔고조회, 매수/매도 주문까지 가능

import requests

접근토큰 발급 후 시세 조회 예시

```
headers = {"Authorization": f"Bearer {access_token}", ...}
```

```
res = requests.get(
```

```
    "https://openapi.koreainvestment.com:9443/uapi/domestic-stock/v1/quotations/inquire-price",
```

```
    headers=headers,
```

```
    params={"fid_input_iscd": "005930"} # 삼성전자
```

```
)
```

```
price = res.json()["output"]["stck_prpr"] # 현재가
```

2. 카카오톡 "나에게 보내기"

- 카카오 디벨로퍼스에서 앱 생성 → 토큰 발급
- send_to_me API 로 본인 카카오톡에 메시지 전송
- 무료

```
def send_kakao(message):  
    url = "https://kapi.kakao.com/v2/api/talk/memo/default/send"  
    headers = {"Authorization": f"Bearer {kakao_token}"}  
    data = {  
        "template_object": json.dumps({  
            "object_type": "text",  
            "text": message,  
            "link": {"web_url": ""}  
        })  
    }  
    requests.post(url, headers=headers, data=data)
```

3. 전체 흐름 예시

```
import time  
  
def monitor():  
    while True:  
        price = get_current_price("005930") # 현재가 조회  
  
        # 조건 체크 (예: 특정가 이하로 떨어지면)  
        if price < 70000:  
            send_kakao(f"⚠️ 삼성전자 현재가 {price:,}원\n 목표가 이하 도달!")  
            break  
  
        time.sleep(60) # 1 분마다 체크  
  
monitor()
```

실행 환경

Mac Mini 서버에서 24 시간 돌리는 방식이 가장 안정적입니다.

항목	내용
언어	Python
실행환경	Mac Mini (상시 가동)
OS 제약	없음 (RESTAPI라 Mac/Windows 모두 OK)
카카오 토큰	약 30일마다 갱신 필요 (자동화 가능)

시작 순서

1. 한국투자증권 계좌 개설 (이미 있으면 패스)
2. KIS Developers 신청 → <https://apiportal.koreainvestment.com>
3. 카카오 디벨로퍼스 앱 생성 → <https://developers.kakao.com>
4. Python 코드 작성 및 테스트
5. Mac Mini 에 배포, 상시 실행

어떤 종목/조건을 모니터링하고 싶으신지 알려주시면 실제 코드 바로 짜드릴 수 있습니다.